

SHUNYA OS

SYSTEM CONSOLIDATION & GITHUB AUDIT

Complete Repository Analysis

Generated: 2026-08-07 21:09 UTC

Commit: 49e7e82 - BATCH-09-10

Confidential — SHUNYA Internal

1. Executive Summary

This consolidated report combines the System Alignment Audit (architecture files) with the Full GitHub Repository Audit. It provides a complete picture of the SHUNYA OS codebase across 122 commits, 10 branches, and approximately 60 MB of source code spanning Python backend, TypeScript/React frontend, and infrastructure.

122	1,569	60 MB	10	15	0
Total Commits	Files	Repo Size	Branches	Green CI Runs	Open Issues

	Value	Metric	Value
	shunya-os/Shunya-OS	Created	2026-07-12
nch	master	Active Branch	master
mits	122	Main Commits (diverged)	4 unique
language	Python (9.1 MB)	Frontend	TypeScript + React (584 MB)
	■ Last 15 runs green	License	None
tection	None	Releases	0 (5 tags)
s	2 (nish, nishesh)	Stars/Forks	0 / 0

2. Git State & Branch Topology

2.1 Branch Inventory

Branch	Last SHA	Protected	Status
master	49e7e82	No	Active — 23 PROD/BATCH commits
main	920882a	No	Stale — 4 FEP commits, diverged
docs	f200828	No	Dormant
feature/alpha-001a-gmail-oauth	39de5b1	No	Incomplete
feature/alpha-001b-gmail-sync	fe0f0a8	No	Incomplete
feature/alpha-001c-document-import	5f153c6	No	Incomplete
feature/alpha-001j-workflow	f42eb27	No	Incomplete
feature/alpha-002a-universal-crm	bed8ea2	No	Incomplete
feature/alpha-003a-dashboard	a78c948	No	Incomplete
legacy/panchi-backend	0f3977b	No	Archived

2.2 master ↔ main Divergence

The two primary branches (master and main) have structurally diverged. master contains 23 PROD batch commits (the complete intelligence pipeline from PROD-04 through BATCH-09-10). main contains 4 FEP-cycle commits (Founder Experience Platform fixes including org creation, API client, and founder signup fixes). The diff spans 734 files with 47,323 insertions and 109,916 deletions — indicating structural reorganization rather than simple conflict.

Key observation: main appears to have been a cleanup/reorganization pass (removed 110K lines), while master has been the active development trunk building the runtime layer. These need reconciliation.

3. Architecture File Analysis

3.1 Core Runtime Files

File	Lines	Role	Key Function(s)
loop.py	189	Autonomous execution engine	run_cycle(), run_loop()
decision_engine.py	239	Pure intelligence layer	get_next_action(), decide_lead_stage(), decide_entity()
entity.py	14	Generic Entity abstraction	Entity(db.Model)
timeline.py	33	Unified entity history	get_entity_timeline()
generator.py	12	Structured output	generate_output()
processor.py	24	Inbound→Message pipeline	process_inbound()
delivery.py	11	Message delivery	deliver_messages()
entity_routes.py	29	REST entity API	list, get, timeline endpoints
webhook_routes.py	19	Ingestion endpoint	POST /api/v1/webhook/

3.2 Communication Layer (15 files, 1,748 lines)

models.py (297 loc) — Message, ExternalMessage, CommunicationSource, OAuthState

runtime.py (214 loc) — Communication runtime orchestration

adapters.py (168 loc) — Provider adapters

adapter.py (94 loc) — Base adapter class

policy.py (157 loc) — Capture policy models

conversation.py (154 loc) — Conversation management

normalizer.py (143 loc) — Message normalization

routes.py (130 loc) — Communication API routes

oauth.py (258 loc) — OAuth flow management

4. GitHub Metadata & Health

4.1 CI/CD Pipeline

Two GitHub Actions workflows: CI (ci.yml) and Dependency Graph. The CI pipeline is healthy — the last 15 runs on master all completed with success. Only 3 historical failures exist, all from the DOC-01 / PROD-99 era before the batch pipeline stabilized.

4.2 Tags & Releases

Tag	SHA	Significance
v0.11.0	74f8374	Latest version tag
v0.1.0-production	d49dc5b	First production tag
v0.1-runtime-stable	f97cfa5	Runtime stability milestone
founder-ready-pre-alpha	64c0cc3	Pre-alpha FEP milestone
canon-v1.0.0	1e58690	Canonical architecture spec

Note: 5 tags exist but 0 formal GitHub Releases. Tags are raw git pointers — no release notes, no binary assets, no release artifacts. This means the project has never formally shipped.

4.3 Risk & Gap Assessment

Severity	Issue	Impact
■ HIGH	No branch protection on master	Direct pushes bypass code review
■ HIGH	No license file	Legal ambiguity — no one can use or contribute
■ MEDIUM	master ↔ main divergence (734 files)	Merge conflicts, lost work risk
■ MEDIUM	Frontend 584 MB vs backend 18 MB	10x imbalance; possible bloat
■ MEDIUM	0 formal GitHub Releases	No shipped milestones despite 5 tags
■ LOW	No open issues or PRs	Clean slate but no tracked work
■ LOW	No topics set	Poor discoverability on GitHub
■ LOW	6 dormant alpha feature branches	Orphaned work, integration burden

5. Key Learnings & Patterns

5.1 Where We Are on the SHUNYA OS Project

SHUNYA OS has been under active development for approximately 27 days. The project has delivered:

A complete intelligence runtime — Object System → Decision Engine → Execution Loop → Commitments → Observations → Lead Lifecycle → Productization Layer.

An entity abstraction layer with generic Entity model and REST API.

A communication layer with Message model, inbound processing, webhook ingestion, and delivery pipeline.

CI/CD pipeline with 15 consecutive green builds.

Frontend (TypeScript/React) infrastructure at 584 MB.

5.2 What Needs to Be Done to Complete the Project

Immediate (must fix before continuing):

Resolve master ↔ main branch divergence — merge or rebase

Add branch protection to master (require PRs + passing CI)

Add a license file (MIT or proprietary)

Create a formal GitHub Release from the latest green commit

Short-term (next batches):

Connect the communication layer to real APIs (WhatsApp, Email)

Implement the message delivery pipeline end-to-end

Add entity-to-entity linking in the execution loop

Clean up or archive dormant alpha feature branches

Reduce frontend bundle size or audit for dead code

Medium-term:

Implement user authentication and multi-tenancy

Build the workspace runtime for end-user interaction

Add monitoring, observability, and production logging

Set up staging/production deployment pipeline

5.3 What Needs to Be Fixed

Severity	Issue	Impact
■	Branch divergence	master and main must converge before further work
■	No license	Legal blocker for any external use or contribution
■	No branch protection	Single-actor workflow accepted but risk grows
■	Zero open issues/PRs	No work tracking — everything in conversation memory
■	Frontend bloat (584 MB)	vs 18 MB backend — investigate
■	6 orphaned alpha branches	Either integrate or archive
■	No topics on repo	Easy fix for discoverability

6. Pattern Analysis: Are We in a Rebuild Loop?

6.1 Evidence of Parallel Workstreams

The repository shows clear evidence of **two parallel development streams** that never converged:

Stream A (master): The PROD batch pipeline — a systematic, bottom-up build of the intelligence stack (Object System → Decision Engine → Loop → Commitments → Observations → Leads → Productization). This stream is well-structured, sequentially ordered, and produces green CI.

Stream B (main — FEP Cycle): The Founder Experience Platform — a top-down UX/build focused on founder signup, authentication, workspace rendering, and frontend integration. This stream was reorganized (110K lines removed) and had bug fixes that master doesn't have.

6.2 The Pattern That Concerns Me

The evidence suggests a **fork-and-replace pattern** rather than iterative convergence:

1. **main was reorganized aggressively** — 110K lines removed, suggesting a belief that the codebase needed cleanup rather than extension. This is often a symptom of architecture dissatisfaction.
2. **master built the runtime independently** — 23 commits of pure runtime construction. The FEP fixes on main (org creation, auth) were never ported to master.
3. **6 alpha feature branches are dormant** — each represents a feature that was started but either never finished or never integrated. This indicates feature-scope creep.
4. **0 formal releases despite 5 tags** — work is being done and tagged but never shipped. The project accumulates without publishing.
5. **584 MB frontend** — the frontend is 32x the backend size. This is unusually large for a React/TypeScript project and may contain multiple generations of frontend builds.

6.3 Are We Stuck?

The project is **not stuck in a loop of rebuilding ShunyaOS from scratch** — the PROD batch progression is linear and additive. However, there are warning signs:

Divergent branches — if you switch between master and main frequently, you're effectively maintaining two versions of the same system. The 734-file diff means they've already diverged past easy reconciliation.

Feature branches that never merge — 6 alpha branches all started but never completed. If this pattern continues, the project accumulates 'graveyard' branches instead of converging.

No formal release cadence — without releases, there's no forcing function to resolve divergence, clean up branches, or stabilize. Work continues indefinitely without shipping.

6.4 Recommended Next Actions

1. **Converge branches NOW.** Merge main into master (or rebase). Get to a single truth before writing another line of code. This is the highest-risk item.
2. **Publish Release v0.12.0** from the current master HEAD (49e7e82). Create a formal GitHub Release with release notes. This anchors the project.
3. **Decide on alpha branches.** For each of the 6 alpha branches: merge, close, or archive. Do not leave them hanging.
4. **Add license + branch protection.** These are quick wins that prevent future problems.
5. **Audit frontend size.** 584 MB is suspicious — check for duplicate node_modules, old build artifacts, or oversized assets.

Core Insight: SHUNYA OS has a solid, well-structured backend runtime that is production-viable. The risk is not in the code quality but in the *organizational patterns* — branch divergence, unreconciled workstreams, and lack of formal shipping discipline. The PROD batch methodology is working. The FEP cycle provided valuable UX feedback that needs integration. Merge the streams, anchor with a release, and continue.

7. Architecture File Contents

Full source code of all audited architecture files for reference.

File: loop.py (189 lines, 6827 bytes)

```
"""Continuous execution loop – observes objects and commitments, determines next action, executes, persists.
Emits pure observation signals: state_changed after update, no_action on noop.
Stateless, deterministic, no memory across cycles.
PROD-13: Relational execution graph propagation.
After a successful update, the loop checks for outbound relations and
propagates execution triggers to connected objects – one cycle, one chain.
No recursion, no business logic.
PROD-17: Autonomous execution loop – commitments.
The same cycle processes all commitments: gap-aware decision from the
latest observation, then applies it. Continuous processing, one cycle.
"""

import time
import logging
from app import db
from app.objects.models import Object
from app.runtime.decision_engine import get_next_action
from app.execution_engine.engine import execute_action
from app.signals.service import emit_signal
from app.graph.service import get_targets
from app.commitments.models import Commitment
from app.runtime.decision_engine import decide_next_from_commitment
from app.commitments.service import apply_decision
from app.runtime.decision_engine import decide_lead_task
from app.runtime.decision_engine import decide_lead_stage
from app.runtime.decision_engine import decide_entity
from app.communication.service import MessageService
from app.output.generator import generate_output
from app.communication.processor import process_inbound
from app.communication.delivery import deliver_messages
from app.core.entity import Entity
from app.models import Lead
logger = logging.getLogger(__name__)
_LOOP_INTERVAL = 3

def _propagate_to_targets(obj, summary: dict):
    """Propagate execution to all targets declared via object_relations.
    Pure structural propagation: every target gets one decision + execute
    cycle. Noop targets are skipped. Errors are recorded per-target.
    """
    relations = get_targets(obj.id)
    for rel in relations:
        try:
            target = Object.query.get(rel.target_object_id)
            if target is None:
                continue
            target_action = get_next_action(target)
            if target_action.get("type") == "noop":
                emit_signal(target.id, "no_action", {"state": target.state})
                summary["noops"] += 1
                summary["signals_emitted"] += 1
                continue
```

```

        state_before = dict(target.state or {})
        execute_action(target, target_action)
        emit_signal(
            target.id, "state_changed",
            {"from": state_before, "to": target.state},
        )
        summary["actions_taken"] += 1
        summary["signals_emitted"] += 1
    except Exception as e:
        logger.error(
            "Propagation error: source=%d target_id=%s: %s",
            obj.id, rel.target_object_id, e,
        )
        summary["errors"].append(
            {"object_id": rel.target_object_id, "error": str(e)},
        )
def run_cycle() -> dict:
    """Run one iteration of the execution loop.
    For each object:
    1. Determine next action via decision engine
    2. Execute action if update
    3. Propagate to relational targets
    4. Emit pure observation signal
    """
    objects = Object.query.all()
    summary = {
        "total_objects": len(objects),
        "actions_taken": 0,
        "noops": 0,
        "signals_emitted": 0,
        "errors": [],
    }
    for obj in objects:
        try:
            action = get_next_action(obj)
            if action["type"] == "noop":
                emit_signal(obj.id, "no_action", {"state": obj.state})
                summary["noops"] += 1
                summary["signals_emitted"] += 1
                continue
            state_before = dict(obj.state or {})
            execute_action(obj, action)
            emit_signal(
                obj.id, "state_changed",
                {"from": state_before, "to": obj.state},
            )
            summary["actions_taken"] += 1
            summary["signals_emitted"] += 1
            # PROD-52: after state update – generate output and create message
            output = generate_output(obj)
            MessageService.create(
                entity_id=obj.id,
                content=str(output),
            )

```

```

        direction="outbound",
        channel="system"
    )
    # PROD-13: propagate to relational targets
    _propagate_to_targets(obj, summary)
except Exception as e:
    logger.error("Loop error on object %d: %s", obj.id, e)
    summary["errors"].append({"object_id": obj.id, "error": str(e)})
# PROD-17: process all commitments – gap-aware decision → apply
commitments = Commitment.query.all()
for c in commitments:
    decision = decide_next_from_commitment(c)
    apply_decision(c, decision)
# PROD-28/30: process leads – task creation + outcome
leads = Lead.query.all()
for lead in leads:
    task_dec = decide_lead_task(lead)
    if task_dec.get("type") == "update":
        lead.outcome = "attempted"
    # PROD-32-34: stage progression
    stage_dec = decide_lead_stage(lead)
    # PROD-40: support multi-action list decisions
    if isinstance(stage_dec, list):
        for dec in stage_dec:
            if dec.get("type") == "update":
                for k, v in dec.get("payload", {}).items():
                    setattr(lead, k, v)
            elif stage_dec.get("type") == "update":
                for k, v in stage_dec.get("payload", {}).items():
                    setattr(lead, k, v)
# PROD-59: process inbound events before entity processing
process_inbound()
# PROD-45: process all Entities – generic state transitions
entities = Entity.query.all()
for e in entities:
    ed = decide_entity(e)
    if ed.get("type") == "update":
        for k, v in ed.get("payload", {}).items():
            setattr(e, k, v)
# PROD-59: deliver pending messages at end of cycle
deliver_messages()
db.session.commit()
return summary

def run_loop(interval: int = _LOOP_INTERVAL, cycles: int = None):
    """Run the execution loop continuously."""
    count = 0
    while cycles is None or count < cycles:
        count += 1
        try:
            summary = run_cycle()
            if summary["actions_taken"] > 0 or summary["signals_emitted"] > 0:
                logger.info(
                    "Loop cycle %d: %d objects, %d actions, %d signals",
                    count, summary["total_objects"],
                    summary["actions_taken"], summary["signals_emitted"],
                )
        except Exception as e:
            logger.error("Loop cycle %d failed: %s", count, e)
            time.sleep(interval)

```

File: decision_engine.py (239 lines, 7216 bytes)

```
"""Decision Engine – pure intelligence, returns next action for an object.
Supports multi-field payload evolution and structural multi-object intelligence.
No business assumptions – only structure, not meaning.
PROD-12: Single-link interaction (foundation).
An object may reference one other object via its `linked_to` state field.
When the referenced object reaches version 2, the linking object is synced.
PROD-13: Full object graph awareness.
An object may declare multiple structural relations:
{
    "relations": [
        {"type": "depends_on", "target_id": 12},
        {"type": "blocks", "target_id": 15}
    ]
}
Each relation is resolved when its target object reaches version 2.
Resolved relations accumulate in `resolved_relations` (non-destructive).
Once all declared relations are resolved, the object itself progresses to
version 2 so downstream objects can depend on it – graph propagation.
Relations are pure structure: {type, target_id}. No business meaning.
"""

from app.objects.models import Object
from app.commitments.models import Commitment
from app.observations.models import Observation
from app.models import Task
def build_context(entity):
    """Build a decision context dict for a lead/entity."""
    return {
        "state": getattr(entity, "stage", None),
        "outcome": getattr(entity, "outcome", None),
        "tasks": [],
        "observations": []
    }
def get_next_action(obj) -> dict:
    """Decide the next action for an object based purely on its state.
    Args:
        obj: Object instance with .state dict.
    Returns:
        update with payload if state progression needed.
        noop if state is fully evolved.
    """
    state = obj.state or {}
    if not state:
        return {
            "type": "update",
            "payload": {
                "initialized": True,
                "version": 1
            }
        }
```

```

    }
# PROD-13 – structural multi-object graph awareness.
# Resolve every declared relation whose target has reached version 2.
# resolved_relations accumulates across cycles (non-destructive).
if state.get("relations"):
    resolved = set(state.get("resolved_relations") or [])
    newly = []
    for rel in state["relations"]:
        if not isinstance(rel, dict):
            continue
        target_id = rel.get("target_id")
        if target_id is None or target_id in resolved:
            continue
        target = Object.query.get(target_id)
        if target is not None and (target.state or {}).get("version") == 2:
            newly.append(target_id)
    if newly:
        accumulated = sorted(resolved | set(newly))
        return {
            "type": "update",
            "payload": {
                "resolved_relations": accumulated
            }
        }
# All declared relations resolved → progress this object to version 2
# so downstream objects can depend on it (graph propagation).
if not state.get("initialized"):
    return {
        "type": "update",
        "payload": {
            "initialized": True,
            "version": 1
        }
    }
if state.get("version") == 1:
    return {
        "type": "update",
        "payload": {
            "version": 2
        }
    }
}

# PROD-12 – structural single-link interaction (preserved).
# If this object references another object, and that object has reached
# version 2, bring this object into sync. No business logic.
if state.get("linked_to"):
    linked = Object.query.get(state["linked_to"])
    if linked is not None and (linked.state or {}).get("version") == 2:
        return {
            "type": "update",
            "payload": {
                "synced": True
            }
        }
}

if state.get("initialized") and state.get("version") == 1:
    return {

```

```

        "type": "update",
        "payload": {
            "version": 2
        }
    }
}
return {"type": "noop"}
def decide_next_from_commitment(commitment: Commitment):
    """
    Decision based on latest observation.
    Returns:
        update_commitment with status=completed if latest observation
        is matched; noop otherwise.
    """
    # fetch latest observation
    obs = (
        Observation.query
        .filter_by(commitment_id=commitment.id)
        .order_by(Observation.id.desc())
        .first()
    )
    # no observation yet
    if not obs:
        return {"type": "noop"}
    # if matched → complete commitment
    if obs.status == "matched":
        return {
            "type": "update_commitment",
            "payload": {"status": "completed"}
        }
    # if deviated → mark failure
    if obs.status == "deviated":
        return {
            "type": "update_commitment",
            "payload": {"status": "failed"}
        }
    return {"type": "noop"}
def decide_lead_task(lead):
    """If a lead has no tasks, return a task creation action."""
    task_count = Task.query.filter_by(lead_id=lead.id).count()
    if task_count == 0:
        return {
            "type": "update",
            "payload": {"task": "Call customer"}
        }
    return {"type": "noop"}
def decide_lead_stage(lead):
    """Progress lead through lifecycle stages based on outcome.

```

```

Uses build_context for unified context access.
"""
context = build_context(lead)
# PROD-39: multi-step decision for quoted leads
if context["state"] == "quoted" and context["outcome"] != "closed":
    return [
        {"type": "update", "payload": {"task": "Follow up"}},
        {"type": "update", "payload": {"priority": "high"}}
    ]
# RULE: contacted with no tasks → create "Send quote"
if context["state"] == "contacted":
    task_count = Task.query.filter_by(lead_id=lead.id).count()
    if task_count == 0:
        return {
            "type": "update",
            "payload": {"task": "Send quote"}
        }
if not context["outcome"]:
    return {"type": "noop"}
if context["outcome"] == "attempted":
    if context["state"] == "new":
        return {
            "type": "update",
            "payload": {"stage": "contacted"}
        }
    if context["state"] == "contacted":
        return {
            "type": "update",
            "payload": {"stage": "quoted"}
        }
    if context["state"] == "quoted":
        return {
            "type": "update",
            "payload": {"stage": "closed"}
        }
    return {"type": "noop"}
def decide_entity(entity):
    """Generic decision for an Entity based on its state."""
    if entity.state == "new":
        return {"type": "update", "payload": {"state": "in_progress"}}
    return {"type": "noop"}
def explain_decision(entity, decision):
    return {
        "entity_id": entity.id,
        "decision": decision,
        "reason": "Derived from current state and context",
        "state": entity.state
    }
}

```

File: entity.py (14 lines, 402 bytes)

```
"""Generic Entity base model – abstraction layer for any business entity."""
from app import db
class Entity(db.Model):
    __tablename__ = "entities"
    id = db.Column(db.Integer, primary_key=True)
    type = db.Column(db.String(50))
    state = db.Column(db.String(50))
    data = db.Column(db.JSON)
    def __repr__(self):
        return f"<Entity #{self.id} [{self.type}] state={self.state}>"
```


File: processor.py (24 lines, 648 bytes)

```
from app.communication.inbound import InboundEvent
from app.communication.models import Message
from app import db

def process_inbound():
    events = InboundEvent.query.filter_by(processed=False).all()
    for e in events:
        # naive mapping (will evolve later)
        entity_id = e.payload.get("entity_id")
        if entity_id:
            msg = Message(
                entity_id=entity_id,
                content=str(e.payload),
                direction="inbound",
                channel=e.source,
                status="received"
            )
            db.session.add(msg)
        e.processed = True
    db.session.commit()
```

File: generator.py (12 lines, 310 bytes)

```
def generate_output(entity):
    state = entity.state or {}
    if entity.type == "lead":
        return {
            "summary": f"Lead at stage: {state.get('stage')}",
            "next_action": state.get("next_step")
        }
    return {
        "summary": "Generic entity",
        "state": state
    }
```

File: timeline.py (33 lines, 918 bytes)

```
from app.models import Task
from app.observations.models import Observation
from app.communication.models import Message
def get_entity_timeline(entity_id):
    events = []
    tasks = Task.query.filter_by(entity_id=entity_id).all()
    observations = Observation.query.filter_by(entity_id=entity_id).all()
    messages = Message.query.filter_by(entity_id=entity_id).all()
    for t in tasks:
        events.append({
            "type": "task",
            "data": t.description,
            "created_at": t.created_at
        })
    for o in observations:
        events.append({
            "type": "observation",
            "data": o.status,
            "created_at": o.created_at
        })
    for m in messages:
        events.append({
            "type": "message",
            "data": m.content,
            "created_at": m.created_at
        })
    return sorted(events, key=lambda x: x["created_at"])
```

File: entity_routes.py (29 lines, 792 bytes)

```
from flask import Blueprint, jsonify
from app.core.entity import Entity
from app.core.timeline import get_entity_timeline
entity_bp = Blueprint("entity_api", __name__, url_prefix="/api/v1/entities")
@entity_bp.route("/", methods=["GET"])
def list_entities():
    entities = Entity.query.all()
    return jsonify([
        {"id": e.id, "type": e.type, "state": e.state}
        for e in entities
    ])
@entity_bp.route("/<int:entity_id>", methods=["GET"])
def get_entity(entity_id):
    e = Entity.query.get_or_404(entity_id)
    return jsonify({
        "id": e.id,
        "type": e.type,
        "state": e.state,
        "data": e.data
    })
@entity_bp.route("/<int:entity_id>/timeline", methods=["GET"])
def timeline(entity_id):
    return jsonify(get_entity_timeline(entity_id))
```

File: webhook_routes.py (19 lines, 476 bytes)

```
from flask import Blueprint, request, jsonify
from app.communication.inbound import InboundEvent
from app import db
webhook_bp = Blueprint("webhook_api", __name__, url_prefix="/api/v1/webhook")
@webhook_bp.route("/", methods=["POST"])
def ingest():
    data = request.json or {}
    event = InboundEvent(
        source=data.get("source", "unknown"),
        payload=data
    )
    db.session.add(event)
    db.session.commit()
    return jsonify({"status": "received"})
```